

Laporan Praktikum Pekan 4

“PHP - MySQL(PDO)”

Disusun Untuk Memenuhi Tugas Mata Kuliah Pemrograman Web

DOSEN PENGAMPU:

ARIFAN RAHMAN S.Kom, M.Kom



DISUSUN OLEH:

Karimah Irsyadiyah (2411533018)

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
T.A 2025/2026**

Daftar Isi

BAB I PENDAHULUAN.....	2
1.1 Latar Belakang.....	2
1.2 Tujuan Praktikum.....	3
LINK GITHUB:.....	3
BAB II LANGKAH-LANGKAH PRAKTIKUM.....	3
2.1 Connection PHP to MySQL.....	3
2.2 Create Database	4
2.3 Create Table	5
2.4 Insert Data.....	7
2.5 Insert Multiple Data	8
2.6 Select Data	9
LATIHAN PRAKTIKUM: CRUD MyGuests Menggunakan PDO	10
BAB III PENUTUP	18
3.1 Kesimpulan	18

BAB I PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan aplikasi web dinamis, database berperan penting sebagai media penyimpanan data. PHP merupakan salah satu bahasa pemrograman server-side yang sering digunakan untuk mengelola data pada database MySQL. Salah satu cara yang

direkomendasikan untuk menghubungkan PHP dengan MySQL adalah menggunakan PDO (PHP Data Objects).

PDO menyediakan antarmuka yang lebih aman, fleksibel, dan mendukung berbagai jenis database. Dengan menggunakan PDO, proses manipulasi data seperti menambah, mengubah, menghapus, dan menampilkan data dapat dilakukan dengan lebih mudah dan aman melalui prepared statement.

Pada praktikum ini dipelajari cara menghubungkan PHP dengan MySQL menggunakan PDO, membuat database, membuat tabel, melakukan operasi insert data, prepared statement, menampilkan data, hingga membangun aplikasi CRUD

1.2 Tujuan Praktikum

1. Memahami konsep koneksi PHP dengan MySQL menggunakan PDO.
2. Mempelajari pembuatan database menggunakan PHP.
3. Mempelajari pembuatan tabel menggunakan PHP.
4. Memahami proses insert data ke database.
5. Memahami penggunaan transaction pada PDO.
6. Memahami penggunaan prepared statement.
7. Menampilkan data dari database menggunakan PHP.
8. Membuat aplikasi CRUD sederhana menggunakan PDO.

LINK GITHUB:

<https://github.com/iMalrsdyh/Pemrograman-Web-PHP-Native/tree/main/lab04>

BAB II

LANGKAH-LANGKAH PRAKTIKUM

2.1 Connection PHP to MySQL

File: connection.php

```
<?php
$servername = "localhost";
$username = "root";
$password = "";
$port = 3306;
```

```

try {
    $conn = new PDO("mysql:host=$servername;port=$port", $username,
$password);
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    echo "Connected successfully";
} catch (PDOException $e) {
    echo "Connection failed: " . $e->getMessage();
}

$conn = null;
?>

```

Penjelasan:

Pada tahap ini dibuat koneksi antara PHP dan MySQL menggunakan PDO. Parameter yang digunakan meliputi hostname, username, password, dan port database.

Objek PDO digunakan untuk membuat koneksi ke server MySQL. Jika koneksi berhasil maka akan ditampilkan pesan "Connected successfully", sedangkan jika gagal akan menampilkan pesan error.

Fungsi yang digunakan:

- PDO()
- setAttribute()
- PDO::ATTR_ERRMODE
- PDO::ERRMODE_EXCEPTION

Tujuan tahap ini adalah memastikan PHP berhasil terhubung dengan MySQL sebelum melakukan operasi database lainnya.

2.2 Create Database

File: create_db.php

```

<?php
$servername = "localhost";
$username = "root";
$password = "";
$port = 3306;

try {

```

```

$conn = new PDO("mysql:host=$servername;port=$port", $username,
$password);
$conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

$sql = "CREATE DATABASE IF NOT EXISTS my_db";
$conn->exec($sql);

echo "Database created successfully<br>";
} catch (PDOException $e) {
    echo $sql . "<br>" . $e->getMessage();
}

$conn = null;
?>

```

Penjelasan:

Pada tahap ini dibuat database baru bernama my_db menggunakan perintah SQL CREATE DATABASE.

Program menggunakan method exec() untuk menjalankan perintah SQL tanpa mengembalikan data.

Query yang dijalankan:

CREATE DATABASE IF NOT EXISTS my_db

Perintah *IF NOT EXISTS* digunakan agar database tidak dibuat ulang apabila sudah tersedia sebelumnya.

Output yang dihasilkan adalah pesan bahwa database berhasil dibuat.

2.3 Create Table

File:

- db_connection.php

```

<?php
$servername = "localhost";
$username = "root";
$password = "";
$port = 3306;
$dbname = "my_db";

try {

```

```

    $conn = new PDO("mysql:host=$servername;port=$port;dbname=$dbname",
$username, $password);
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    echo "Connection failed: " . $e->getMessage();
}
?>

```

- **create_table.php**

```

<?php
require_once 'db_connection.php';

try {
    $sql = "CREATE TABLE MyGuests (
        id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
        firstname VARCHAR(30) NOT NULL,
        lastname VARCHAR(30) NOT NULL,
        email VARCHAR(50) UNIQUE,
        reg_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
    )";

    $conn->exec($sql);
    echo "Table MyGuests created successfully";
} catch(PDOException $e) {
    echo $sql . "<br>" . $e->getMessage();
}

$conn = null;
?>

```

Penjelasan:

Setelah database berhasil dibuat, langkah berikutnya adalah membuat tabel MyGuests.

Pada file db_connection.php ditambahkan variabel dbname yang berisi nama database yang akan digunakan.

Tabel MyGuests memiliki struktur:

- id
- firstname
- lastname
- email
- reg_date

Kolom id digunakan sebagai Primary Key dan Auto Increment sehingga setiap data yang ditambahkan akan memiliki ID unik secara otomatis.

Kolom reg_date digunakan untuk menyimpan waktu pendaftaran data.

2.4 Insert Data

File: insert_data.php

```
<?php
require_once 'db_connection.php';

try {

    $sql = "INSERT INTO MyGuests (firstname, lastname, email)
    VALUES ('John', 'Doe', 'john@example.com')";

    $conn->exec($sql);

    echo "New record created successfully";

} catch(PDOException $e) {

    echo $sql . "<br>" . $e->getMessage();

}

unset($conn);
?>
```

Penjelasan:

Tahap ini digunakan untuk menambahkan satu data ke tabel MyGuests.

Data yang dimasukkan:

- First Name : John
- Last Name : Doe
- Email : john@example.com

Perintah SQL yang digunakan:

INSERT INTO MyGuests

Setelah query berhasil dijalankan, data akan tersimpan pada database dan muncul pesan bahwa data berhasil ditambahkan.

2.5 Insert Multiple Data

File: insert_multiple.php

```
<?php
require_once 'db_connection.php';

try {
    $conn->beginTransaction();

    $conn->exec("INSERT INTO MyGuests (firstname, lastname, email)
VALUES ('Mary', 'Moe', 'mary@example.com')");

    $conn->exec("INSERT INTO MyGuests (firstname, lastname, email)
VALUES ('Julie', 'Dooley', 'julie@example.com')");

    $conn->exec("INSERT INTO MyGuests (firstname, lastname, email)
VALUES ('Kane', 'Que', 'kane@example.com')");

    $conn->commit();
    echo "New records created successfully";
} catch(PDOException $e) {
    $conn->rollback();
    echo "Error: " . $e->getMessage();
}

$conn = null;
?>
```

Penjelasan:

Pada tahap ini dilakukan penambahan beberapa data sekaligus menggunakan transaction.

Fungsi yang digunakan:

- beginTransaction()
- commit()
- rollback()

Transaction digunakan agar seluruh query dijalankan sebagai satu kesatuan proses.

Apabila salah satu query gagal, maka seluruh perubahan akan dibatalkan menggunakan rollback().

Data yang ditambahkan:

- Mary Moe

- Julie Dooley
- Kane Que

Metode ini menjaga konsistensi data pada database.

2.6 Select Data

File: select_data.php

```
<?php
require_once 'db_connection.php';

try {
    $stmt = $conn->prepare("SELECT * FROM MyGuests");
    $stmt->execute();
    $stmt->setFetchMode(PDO::FETCH_ASSOC);
    $result = $stmt->fetchAll();
} catch(PDOException $e) {
    echo "Error: " . $e->getMessage();
}

$conn = null;
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <h1>My Guests</h1>
    <table border="1" cellpadding="8" cellspacing="0">
        <thead>
            <tr>
                <th>ID</th>
                <th>Firstname</th>
                <th>Lastname</th>
                <th>Registered Date</th>
            </tr>
        </thead>
        <tbody>
            <?php foreach ($result as $guest) { ?>
                <tr>
                    <td><?= $guest['id'] ?></td>
                    <td><?= $guest['firstname'] ?></td>
                    <td><?= $guest['lastname'] ?></td>
```

```

        <td><?= date_format(date_create($guest['reg_date']),
'D, d F Y H:i') ?></td>
    </tr>
    <?php } ?>
</tbody>
</table>
</body>
</html>

```

Penjelasan:

Tahap ini digunakan untuk menampilkan seluruh data pada tabel MyGuests.

Data diambil menggunakan query:

```
SELECT * FROM MyGuests
```

Data yang diperoleh kemudian disimpan ke dalam array menggunakan:

```
fetchAll(PDO::FETCH_ASSOC)
```

Selanjutnya data ditampilkan dalam bentuk tabel HTML yang berisi:

- ID
- First Name
- Last Name
- Registered Date

Tanggal pendaftaran diformat menggunakan fungsi `date_create()` dan `date_format()` agar lebih mudah dibaca.

LATIHAN PRAKTIKUM: CRUD MyGuests Menggunakan PDO

1. db.php

```

<?php
$servername = "localhost";
$username = "root";
$password = "";
$port = 3306;
$dbname = "my_db";

try {
    $connection = new PDO("mysql:host=$servername;port=$port;dbname=$dbname",
$username, $password);
    $connection->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

```

```

} catch (PDOException $e) {
    echo "Connection failed: " . $e->getMessage();
}
?>

```

File ini digunakan untuk membuat koneksi database menggunakan PDO.

Koneksi akan digunakan oleh seluruh fungsi CRUD yang terdapat pada aplikasi.

function.php

```

<?php
/**
 * CREATE
 */
function createGuest($firstname, $lastname, $email)
{
    global $connection;
    $sql = "INSERT INTO myguests (firstname, lastname, email, reg_date)
VALUES (:firstname, :lastname, :email, NOW())";
    $stmt = $connection->prepare($sql);
    $stmt->bindParam(':firstname', $firstname);
    $stmt->bindParam(':lastname', $lastname);
    $stmt->bindParam(':email', $email);

    try {
        $stmt->execute();
        return "Data berhasil ditambahkan!";
    } catch (PDOException $e) {
        return "Gagal menambahkan data: " . $e->getMessage();
    }
}

/**
 * GET ALL DATA
 */
function getGuests()
{
    global $connection;
    $sql = "SELECT * FROM myguests";
    $stmt = $connection->query($sql);
    return $stmt->fetchAll(PDO::FETCH_ASSOC);
}

/**
 * GET DATA BY ID
 */
function getGuest($id)
{
    global $connection;
    $stmt = $connection->prepare("SELECT * FROM myguests WHERE id = :id");
    $stmt->bindParam(':id', $id);
    $stmt->execute();
    return $stmt->fetch(PDO::FETCH_ASSOC);
}

/**

```

```

* UPDATE
*/
function updateGuest($id, $firstname, $lastname, $email)
{
    global $connection;
    $sql = "UPDATE myguests
        SET firstname = :firstname, lastname = :lastname, email =
:email
        WHERE id = :id";

    $stmt = $connection->prepare($sql);
    $stmt->bindParam(':firstname', $firstname);
    $stmt->bindParam(':lastname', $lastname);
    $stmt->bindParam(':email', $email);
    $stmt->bindParam(':id', $id);

    try {
        $stmt->execute();
        return "Data berhasil diupdate!";
    } catch (PDOException $e) {
        return "Gagal mengupdate data: " . $e->getMessage();
    }
}

/**
* DELETE
*/
function deleteGuest($id)
{
    global $connection;
    $sql = "DELETE FROM myguests WHERE id = :id";
    $stmt = $connection->prepare($sql);
    $stmt->bindParam(':id', $id);

    try {
        $stmt->execute();
        return "Data berhasil dihapus!";
    } catch (PDOException $e) {
        return "Gagal menghapus data: " . $e->getMessage();
    }
}
?>

```

File ini berisi kumpulan fungsi CRUD.

createGuest(): Digunakan untuk menambahkan data tamu baru ke database.

getGuests(): Digunakan untuk mengambil seluruh data tamu dari database.

getGuest(): Digunakan untuk mengambil satu data tamu berdasarkan ID.

updateGuest(): Digunakan untuk memperbarui data tamu yang sudah ada.

deleteGuest(): Digunakan untuk menghapus data tamu berdasarkan ID.

2. index.php

File ini merupakan halaman utama aplikasi CRUD.

```
<?php
include 'db.php';
include 'function.php';

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    if (isset($_POST['create'])) {
        $firstname = $_POST['firstname'];
        $lastname = $_POST['lastname'];
        $email = $_POST['email'];
        echo createGuest($firstname, $lastname, $email);
    }

    if (isset($_POST['update'])) {
        $id = $_POST['id'];
        $firstname = $_POST['firstname'];
        $lastname = $_POST['lastname'];
        $email = $_POST['email'];
        echo updateGuest($id, $firstname, $lastname, $email);
    }

    if (isset($_POST['delete'])) {
        $id = $_POST['id'];
        echo deleteGuest($id);
    }
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Guest CRUD</title>
</head>
<body>
    <h2>Tambah Tamu</h2>
    <form method="POST">
        <input type="text" name="firstname" placeholder="First Name"
required><br>
        <input type="text" name="lastname" placeholder="Last Name"
required><br>
        <input type="email" name="email" placeholder="Email" required><br>
        <button type="submit" name="create">Tambah</button>
    </form>

    <h2>Daftar Tamu</h2>
    <table border="1" cellpadding="8" cellspacing="0">
        <thead>
            <tr>
                <th>ID</th>
                <th>First Name</th>
                <th>Last Name</th>
                <th>Email</th>
            </tr>
        </thead>
    </table>
```

```

        <th>Reg Date</th>
        <th>Aksi</th>
    </tr>
</thead>
<tbody>
    <?php
    $guests = getGuests();
    foreach ($guests as $guest) {
    ?>
    <tr>
        <td><?= $guest['id'] ?></td>
        <td><?= $guest['firstname'] ?></td>
        <td><?= $guest['lastname'] ?></td>
        <td><?= $guest['email'] ?></td>
        <td><?= date_format(date_create($guest['reg_date']), 'D, d F Y
H:i') ?></td>
        <td>
            <a href="update.php?id=<?= $guest['id'] ?>">Edit</a>
            <form method="POST" style="display:inline;">
                <input type="hidden" name="id" value="<?= $guest['id']
?>">
                <button type="submit" name="delete" onclick="return
confirm('Are you sure?')">Delete</button>
            </form>
        </td>
    </tr>
    <?php } ?>
</tbody>
</table>
</body>
</html>

```

Data berhasil ditambahkan!

Tambah Tamu

First Name
Last Name
Email
Tambah

Daftar Tamu

ID	First Name	Last Name	Email	Reg Date	Aksi
1	HAI	IYA	HAI@gmail.com	Sat, 20 June 2026 20:51	Edit <button>Delete</button>
2	Karimah	Irsyadiyah	karimah@gmail.com	Sat, 20 June 2026 20:24	Edit <button>Delete</button>
3	wdwd	wdwd	ada@gmail.com	Sat, 20 June 2026 20:52	Edit <button>Delete</button>

Data berhasil dihapus!

Tambah Tamu

First Name
Last Name
Email
Tambah

Daftar Tamu

ID	First Name	Last Name	Email	Reg Date	Aksi
2	Karimah	Irsyadiyah	karimah@gmail.com	Sat, 20 June 2026 20:24	Edit Delete
3	wdwd	wdwd	ada@gmail.com	Sat, 20 June 2026 20:52	Edit Delete

Fitur yang tersedia:

1. Menambahkan data tamu baru.
2. Menampilkan seluruh data tamu.
3. Menghapus data tamu.
4. Menyediakan tombol edit untuk memperbarui data.

Data ditampilkan dalam bentuk tabel yang berisi:

- ID
- First Name
- Last Name
- Email
- Registration Date
- Aksi

3. update.php

```
<?php
include 'db.php';
include 'function.php';
```

```

if (!isset($_GET['id'])) {
    die("ID tamu tidak ditemukan.");
}

$id = $_GET['id'];
$guest = getGuest($id);

if (!$guest) {
    die("Tamu dengan ID tersebut tidak ditemukan.");
}

if ($_SERVER['REQUEST_METHOD'] == 'POST' && isset($_POST['update'])) {
    $firstname = $_POST['firstname'];
    $lastname = $_POST['lastname'];
    $email = $_POST['email'];

    echo updateGuest($id, $firstname, $lastname, $email);
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Update Guest</title>
</head>
<body>
    <h2>Update Tamu</h2>
    <form method="POST">
        <input type="text" name="firstname" value="<?= $guest['firstname'];
?>" placeholder="First Name" required><br>
        <input type="text" name="lastname" value="<?= $guest['lastname']; ?>"
placeholder="Last Name" required><br>
        <input type="email" name="email" value="<?= $guest['email']; ?>"
placeholder="Email" required><br>
        <button type="submit" name="update">Update</button>
    </form>
    <p><a href="index.php">Kembali ke Daftar Tamu</a></p>
</body>
</html>

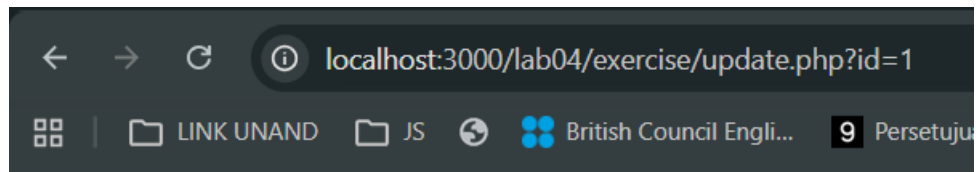
```

File ini digunakan untuk memperbarui data tamu.

Langkah yang dilakukan:

1. Mengambil ID tamu dari URL.
2. Mengambil data tamu dari database.

3. Menampilkan data pada form.
4. Menyimpan perubahan data menggunakan fungsi updateGuest().



Update Tamu

HAI
IYA
HAI@gmail.com
Update

[Kembali ke Daftar Tamu](#)

Data berhasil diupdate!

Tambah Tamu

First Name
Last Name
Email
Tambah

Daftar Tamu

ID	First Name	Last Name	Email	Reg Date	Aksi
1	HAI	IYA	HAI@gmail.com	Sat, 20 June 2026 20:51	Edit <button>Delete</button>
2	Karimah	Irsyadiyah	karimah@gmail.com	Sat, 20 June 2026 20:24	Edit <button>Delete</button>

BAB III

PENUTUP

3.1 Kesimpulan

Pada praktikum ini telah dipelajari penggunaan PHP dengan MySQL menggunakan PDO. Praktikum dimulai dari pembuatan koneksi database, pembuatan database, pembuatan tabel, penambahan data, penggunaan transaction, prepared statement, hingga menampilkan data dari database.

Berhasil dibuat aplikasi CRUD sederhana yang mampu melakukan operasi Create, Read, Update, dan Delete terhadap data tamu pada tabel MyGuests.

Penggunaan PDO memberikan keamanan yang lebih baik terhadap aplikasi web karena mendukung prepared statement sehingga dapat mengurangi risiko SQL Injection.